

PROJECT HANDBOOK

A Plain-Language Guide to Your Telco Churn Prediction Project

This handbook is a companion to your internship report. It explains, in plain language, what each part of the project does, why specific choices were made (which model, which metrics, which features), and how to talk about the project confidently in a viva, interview, or presentation. It does not contain any new analysis — every number here matches your report and notebook.

Read this top to bottom once, then use it as a quick-reference whenever someone asks "why did you do X?"

1. The Big Picture — What Problem Are We Solving?

A telecom company (we called it "ConnectTel") loses customers every month — this is called churn. Roughly 1 in 4 customers (26.5%) eventually cancels their service. The company wants to know, in advance, which customers are likely to leave so the retention team can offer them a discount or a better plan before they cancel.

Your project builds a machine learning model that looks at a customer's details — how long they've been a customer, what plan they're on, how much they pay, what services they use — and predicts whether they are likely to churn.

Q: Why does this matter to a business?

Because keeping an existing customer is 5–7 times cheaper than acquiring a new one. If the company can correctly identify even 80% of the customers who are about to leave, it can target retention offers only at those people instead of wasting money on everyone — saving a large amount of money. Our financial simulation (Section 8 of this handbook) puts a real number on this.

2. The Data — What Are We Working With?

We used the "Telco Customer Churn" dataset (WA_Fn-UseC_-Telco-Customer-Churn.csv) — a well-known, publicly available dataset with 7,043 customers and 21 columns, including a "Churn" column (Yes/No) which is what we're trying to predict.

What's in the data?

- Demographics: gender, senior citizen status, partner, dependents
- Account info: how long they've been a customer (tenure), contract type, payment method, paperless billing
- Services: phone, internet type, and 8 optional add-ons (security, backup, tech support, streaming, etc.)
- Billing: monthly charges and total charges

Q: Why did we clean the data first?

Two reasons. First, the "TotalCharges" column had 11 blank entries — these turned out to belong to brand-new customers (tenure = 0) who simply hadn't been billed yet, so we correctly filled these with 0 rather than guessing. Second, we removed the "customerID" column because it's just a unique label for each row — it has zero predictive value, and if we left it in, one-hot encoding would have created thousands of useless columns (one per customer).

Q: Why is the 26.5% churn rate important?

This is called "class imbalance." If a lazy model just predicted "No churn" for every single customer, it would be right 73.5% of the time — looking "accurate" while being completely useless. This is THE single most important fact that shapes almost every other decision in the project: which metrics we use, how we configure the models, and how we split the data.

3. Exploratory Data Analysis (EDA) — What Did We Discover?

Before building any model, we explored the data to find patterns. Think of this as "getting to know your customer." Here's what we found, ranked roughly by importance:

Finding	The Numbers	What It Means
Contract type is huge	Month-to-month: 42.7% churn Two-year: 2.8% churn	A customer with no long-term commitment is ~15× more likely to leave. This is the single strongest signal in the whole dataset.
Tenure (loyalty) matters a lot	First year: 47.4% churn After 5 years: 6.6% churn	Almost half of brand-new customers churn within their first year. Risk drops steadily the longer someone stays.
The two combine (compound)	Month-to-month + Fiber: 54.6% churn Two-year + No internet: 0.8% churn	The worst-case combination (new contract type + fiber) is over 10× riskier than the best-case combination.
Fiber-optic internet is risky	41.9% churn (vs. 19% for DSL, 7.4% for none)	Counter-intuitive at first — fiber is the "better" service. But it's also the most expensive, attracting price-sensitive customers.
Payment method matters	Electronic check: 45.3% churn Auto-pay methods: 15–17% churn	Customers who pay manually each month are far more likely to leave than those on autopay — autopay signals commitment and reduces "forgot to renew" cancellations.
Bundled services protect	0 services: 43.8% churn All 8 services: 5.3% churn	The more services a customer relies on, the more "switching cost" they face if they leave — strongly protective.
Household ties protect	No partner: 33.0% vs. With partner: 19.7% No dependents: 31.3% vs. With dependents: 15.5%	Customers who share the account with family are stickier — switching providers is a bigger hassle for a household.
Gender doesn't matter	Female: 26.9% vs. Male: 26.2%	Almost no difference — gender is not a useful predictor, and the model correctly learns to ignore it.

Q: Why did we run formal statistical tests (chi-square / t-tests) instead of just trusting the charts?

Charts can be misleading — a difference that looks big on a bar chart might just be random noise, especially with smaller subgroups. We ran six formal hypothesis tests (chi-square for category-vs-category relationships, Welch's t-test for numeric comparisons) at the standard 5% significance threshold. All six came back strongly significant (p-values far below 0.05), which gives us mathematical confidence that these patterns are real and not coincidence. This is what separates a rigorous project from "I made some charts and they looked interesting."

4. Feature Engineering — Why Did We Create New Columns?

"Feature engineering" means creating new columns from the existing ones to help the model see patterns more easily. We created 10 new features. Here's the plain-language reasoning for each:

Feature	Why we made it
ContractRiskScore	Turns "Month-to-month / One year / Two year" into a number (2 / 1 / 0) so the model understands it as a sliding scale of risk, not just three unrelated categories.
LoyaltyScore	Combines tenure AND contract type into one number (long tenure + long contract = high loyalty). This turned out to be one of the most powerful features in the entire model — it captures "how locked-in is this customer" in a single number.
ServiceCount / BundleScore	Counts how many of the 8 optional services a customer uses. Captures the "bundling = stickiness" pattern we saw in the EDA.
TotalChargesPerTenure / AverageMonthlySpend	A customer's lifetime spend divided by how long they've been with us — tells us their "true" average monthly value, smoothing out promotional discounts in early months.
LongTermCustomer	A simple yes/no flag for "has this customer been with us more than a year?" — makes the strong tenure-12-month threshold explicit for the model.
PremiumServiceFlag / PremiumCustomerFlag	Flags customers on fiber + streaming services, or high-spend multi-service customers — useful for targeting premium retention offers.
AutoPaymentFlag	Yes/no flag for whether the customer is on automatic payment — directly captures the payment-method pattern from the EDA.

Q: Did the engineered features actually help, or were they just "extra work"?

They genuinely helped. LoyaltyScore became the #1 most important feature in the Random Forest model and the #2 most important driver in the SHAP analysis (Chapter 8). TotalChargesPerTenure and ContractRiskScore also ranked in the top 7. If they hadn't mattered, the models would have ignored them — instead, several of our hand-crafted features outranked the original raw columns.

5. Why These Four Models? (Logistic Regression, Random Forest, XGBoost, SVM)

We didn't pick these models randomly — each one represents a different "philosophy" of how to draw a decision boundary, and comparing them tells a story.

Model	Why we included it	What it's good for
Logistic Regression	The simplest, most interpretable model. Draws a straight-line (linear) boundary between "will churn" and "won't churn."	Gives us "odds ratios" — e.g. "fiber optic customers are 3.8× more likely to churn." Great for explaining results to non-technical people.
Random Forest	Builds hundreds of decision trees and averages their votes. Can capture non-linear patterns and interactions automatically.	Gives us "feature importance" rankings. Turned out to be very precise (few false alarms) but missed half the actual churners — an important finding, not a failure!
XGBoost	A more advanced, "smarter" version of Random Forest that learns from its own mistakes step by step (boosting). Industry standard for this type of problem.	Best overall balance of catching churners (recall) and overall ranking quality (ROC-AUC). This became our champion model after tuning.
SVM (Support Vector Machine)	A completely different mathematical approach — finds the boundary that maximally separates the two classes, using a "kernel trick" to handle curves.	Acts as a sanity check / alternative. Came close to XGBoost on recall, confirming our results aren't a fluke of one particular algorithm.

Q: Why is XGBoost the "champion" if Logistic Regression scored almost as well?

Great question — and an honest one we addressed directly in the report. XGBoost's ROC-AUC (0.847) edged out Logistic Regression (0.844) only slightly, and Logistic Regression actually had a competitive recall too (0.797 vs 0.805). XGBoost wins because it can capture combination effects — like "month-to-month AND fiber AND low tenure is much worse than any one of those alone" — which a straight-line model can't fully represent. But the small gap is itself a useful finding: if a business wanted maximum simplicity and explainability, Logistic Regression would be a very reasonable choice at almost no performance cost.

Q: Why did Random Forest perform so differently from the others (high precision, low recall)?

This was one of the most interesting findings. Random Forest caught only about half (50.3%) of actual churners, but when it DID flag someone, it was right more often (62% precision) than any other model. This means Random Forest is "cautious" — it only raises a flag when it's quite confident. For our business goal (catch as many at-risk customers as possible), this caution is actually a weakness. But in a different scenario — say, if a false alarm was very costly — Random Forest's caution could be an advantage. This is why we compare multiple models instead of just picking one.

6. Why Recall and ROC-AUC, Not Just Accuracy?

This is probably the single most important conceptual point in the whole project, so it's worth spending time on.

The accuracy trap

Imagine a model that always predicts "this customer will NOT churn," no matter what. Because 73.5% of customers genuinely don't churn, this lazy model would be "73.5% accurate" — sounds impressive! But it would be completely useless, because it never identifies a single at-risk customer. This is why we never lead with accuracy.

The metrics we actually care about

- Recall (Sensitivity): "Of all the customers who actually churned, how many did we catch?" Our champion model catches 80.5% of them. This is our #1 priority — a missed churner (false negative) means we lose that customer's entire future revenue.
- Precision: "Of the customers we flagged as risky, how many actually churned?" Ours is around 52% — meaning about half of our "warnings" are false alarms. That's OK, because a false alarm only costs us a small discount offer to someone who would have stayed anyway.
- ROC-AUC: A single number (0 to 1) summarizing how well the model ranks customers from "most likely to churn" to "least likely," regardless of where we set the cutoff. Our champion scores 0.847 — well above the 0.5 you'd get from random guessing.
- F1-score: A balance between precision and recall, useful for quick comparisons but less important than recall for our specific business problem.

Q: Why is a false negative (missed churner) worse than a false positive (false alarm)?

A false positive costs the company a small retention discount given to someone who was going to stay anyway — a minor, bounded cost. A false negative means a customer leaves WITHOUT us ever offering them anything — we lose their entire future subscription revenue. The costs are wildly asymmetric, so our model is deliberately tuned (via `class_weight='balanced'` and `scale_pos_weight`) to lean toward catching more churners, even if it means more false alarms.

7. Making the Model "Explainable" — Odds Ratios, Feature Importance, SHAP

A model that just spits out "70% chance of churn" without explaining WHY is not very useful to a business team. We used three different techniques to "open the black box," and — importantly — they all told a consistent story.

1. Odds Ratios (from Logistic Regression)

An odds ratio tells you how much a factor multiplies your churn risk. A ratio of 3.8 for "Fiber optic" means having fiber roughly multiplies your odds of churning by 3.8×, holding everything else constant. A ratio of 0.23 for "tenure" means each unit increase in tenure cuts your odds to about 23% of what they were.

Q: Why did some odds ratios seem to contradict the EDA (e.g. LoyaltyScore looked "bad" and MonthlyCharges looked "good")?

This is called multicollinearity — several of our features (tenure, LoyaltyScore, ContractRiskScore, TotalChargesPerTenure) are all measuring closely related things. When you put highly correlated variables into a Logistic Regression together, the model can sometimes "split" or even flip the credit between them in confusing ways, even though the OVERALL prediction is still accurate. This is a normal, well-known statistical phenomenon — not a bug. It's exactly why we didn't rely on odds ratios alone, and cross-checked with Random Forest importances and SHAP, which are not confused by this issue.

2. Feature Importance (from Random Forest)

Random Forest tracks how often each feature is used to split the data, and how much that split improves predictions. The top result: LoyaltyScore (our engineered feature) was the single most important column in the entire dataset — ahead of raw tenure, raw charges, everything.

3. SHAP (SHapley Additive exPlanations)

SHAP is the most modern and rigorous explainability technique — it's based on game theory and tells you, for each individual customer, exactly how much each feature pushed their prediction up or down. Across 800 sample customers, the global ranking was: ContractRiskScore (#1), LoyaltyScore (#2), Fiber-optic internet (#3), Electronic-check payment (#4).

Q: Why does it matter that three different methods (odds ratios, RF importance, SHAP) roughly agree?

Because it tells us the model has learned real, sensible patterns about the world — not random noise or "shortcuts" that happen to fit this particular dataset. When contract type, tenure/loyalty, and internet type show up as top drivers across completely different mathematical techniques, we can be confident these are genuine, actionable business insights, not statistical accidents. This is called "triangulation" and it's a hallmark of trustworthy machine learning.

8. The Financial Simulation — Turning Predictions Into Money

A prediction is only useful if it leads to action. So we ran a simple simulation: what if the retention team only contacted the riskiest 20% of customers, based on our model's predictions?

Step	Result
Customers in our test group	1,409
Of those, how many actually churned	374
Top 20% riskiest customers (our target list)	281 customers
Real churners caught in that top 20%	190 (about half of all churners, from just 20% of the list)
Revenue at risk in that group (annual)	\$178,131.60
Assuming 25% of contacted churners are "saved"	47 customers retained
Revenue saved	\$44,064.13
Cost of running the campaign	\$4,215.00
Net profit from this campaign	\$39,849.13

Q: Why only assume a 25% "save rate"? Isn't that pessimistic?

Yes, deliberately so. In real retention campaigns, not every customer who is offered a discount accepts it or changes their mind — 25% is a conservative, realistic industry-style assumption. By using a conservative number and STILL showing a ~\$40,000 net profit on just 1,409 customers, we make the business case more credible. If anything, the real result is likely to be a lower bound — a more generous (but still realistic) save rate would make the numbers even better.

Q: What is the single biggest takeaway from this whole project?

A 26.5% churn problem that initially looks intractable can be substantially de-risked by focusing resources on a relatively small, well-identified slice of the customer base. By targeting only the top 20% highest-risk customers (using our model's ranking), the retention team can catch roughly half of all churners, at a fraction of the cost of a blanket campaign — and the model tells you exactly WHY each customer is at risk (short tenure, month-to-month contract, fiber-optic, etc.), so the retention offer can be tailored accordingly.

9. Quick Glossary — Terms You Might Get Asked About

Term	Plain-language meaning
Churn	A customer cancelling their subscription / leaving the company.
Class imbalance	When one outcome (here, "no churn") is much more common than the other ("churn") — makes plain accuracy misleading.
Feature	A column of data used as an input to the model (e.g., tenure, contract type).
Feature engineering	Creating new, smarter columns from existing ones to help the model find patterns.
One-hot encoding	Converting a category column (e.g., "Contract: Month-to-month/One year/Two year") into separate 0/1 columns the model can understand.
Train/test split	Splitting the data so the model learns from one part (train) and is judged on a part it has never seen (test) — mimics real-world deployment.
Stratified split	A train/test split that preserves the same churn percentage in both halves, so neither half is accidentally easier or harder.
Cross-validation (CV)	Repeating the train/test process multiple times on different slices of the data to make sure results are stable, not lucky.
Hyperparameter	A "setting" of the model (e.g., how many trees, how fast it learns) that we tune to get the best performance.
GridSearchCV	An automated, exhaustive way of trying many combinations of hyperparameters and keeping the best one.
Overfitting	When a model memorises the training data too well and performs worse on new, unseen data. We checked for this by comparing cross-validation scores to test scores — they were close, so we're fine.
ROC curve / AUC	A graph and single score (0–1) measuring how well a model ranks positive vs. negative cases across all possible decision thresholds.
Confusion matrix	A 2×2 table showing exactly how many predictions were correct/incorrect, broken down by type (true positive, false positive, etc.).
SHAP	A technique that explains individual predictions by showing how much each feature contributed, based on game theory.
Odds ratio	How much a factor multiplies your "odds" of an outcome (>1 increases risk, <1 decreases risk).

That's the whole story, end to end: a real business problem — cleaned, imbalanced data — informed EDA with statistical backing — ten thoughtful engineered features — four complementary models compared on the RIGHT metrics — three independent explainability techniques that agree with each other — and a financial simulation that proves the model is worth deploying. If you can walk someone through this handbook section by section, you can defend every decision in your report.